

PADRÃO DE PROJETO COMPORTAMENTAL · GoF

Chain of Responsibility

Cadeia de Responsabilidade

Um pedido passa de mão em mão até alguém saber resolvê-lo.

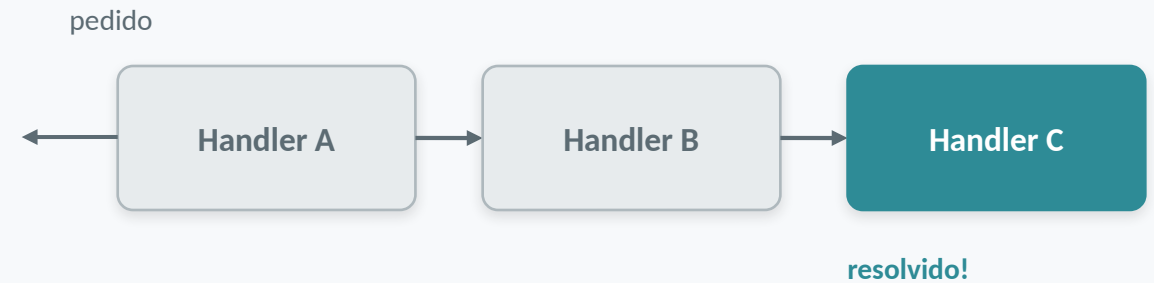




O que é?

É um padrão que **enfileira vários objetos** (handlers). Um pedido entra pelo primeiro da fila:

- se ele sabe resolver, resolve
- se não sabe, empurra para o próximo
- e assim até alguém tratar (ou a fila acabar)

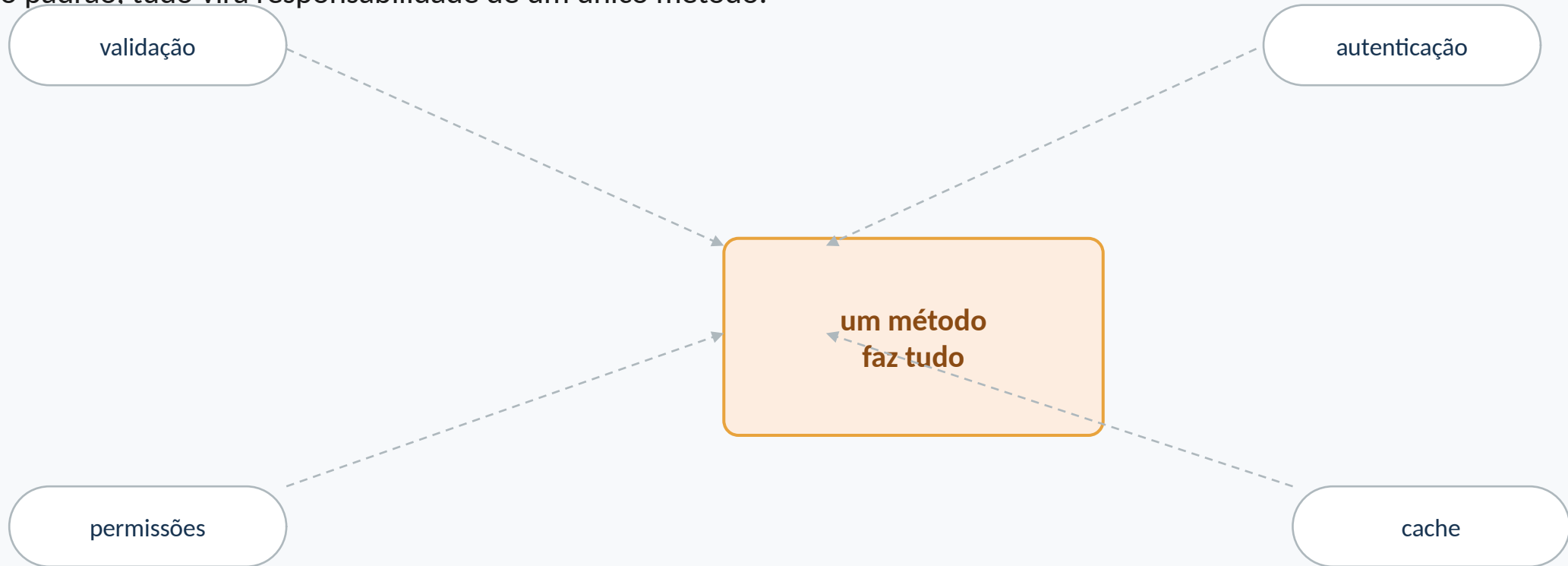


Vantagem: quem envia o pedido nem precisa saber quem, na fila, vai resolvê-lo.



0 problema

Sem o padrão, tudo vira responsabilidade de um único método:



Alterar uma verificação afeta as outras. Reaproveitar uma delas em outro lugar? Quase impossível.



A solução

Cada verificação vira sua própria classe, cuidando de uma única coisa:

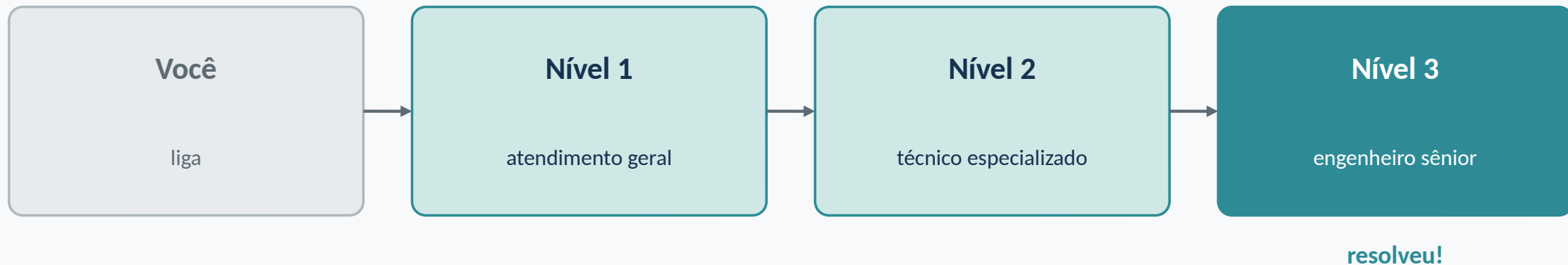


Se um handler não sabe tratar o pedido, ele simplesmente **repassa para o próximo** da fila — sem se preocupar com o que vem depois.



Analogia: central de atendimento

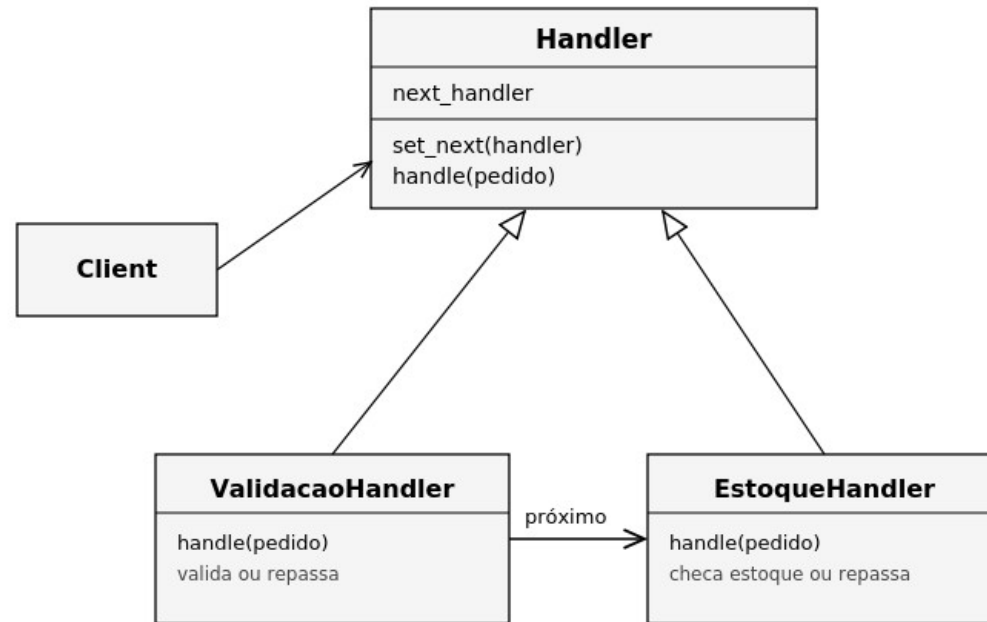
Você liga pro suporte técnico. Cada nível decide: resolve ou transfere.



Assim como os handlers, cada nível decide sozinho — não existe um "chefe" central mandando quem atende o quê.



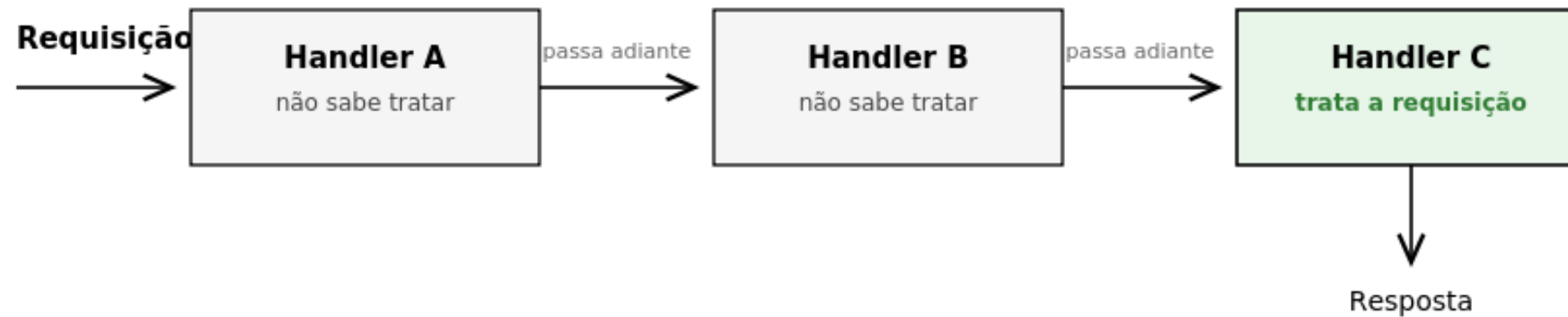
Como funciona



- Handler: guarda o próximo da fila e sabe repassar
- Handlers concretos: cada um cuida de uma verificação
- Client: monta a fila e envia o pedido pro primeiro



O fluxo da requisição



Cada handler decide: trata a requisição ou repassa para o próximo da cadeia

A requisição percorre a cadeia até encontrar um handler capaz de tratá-la — ou chega ao fim sem resposta.



Exemplo em Python

```
class Handler:
    def set_next(self, h):
        self.next_handler = h
        return h

    def handle(self, pedido):
        if self.next_handler:
            return self.next_handler.handle(pedido)

class ValidacaoHandler(Handler):
    def handle(self, pedido):
        if not pedido["valido"]:
            return "Invalido!"
        return super().handle(pedido)
```

A ideia:

toda classe herda de `Handler` e só sabe fazer uma pergunta.

Se não souber responder, chama `super().handle()` e passa a bola pra frente.



Quando usar · cuidados

Use quando

- Não se sabe de antemão quantos handlers serão precisos
- A ordem de tratamento é importante
- A lista de handlers pode mudar em tempo de execução

Cuidados

- Um pedido pode chegar ao fim da fila sem ser tratado
- Cadeias longas ficam mais difíceis de depurar
- Viola o Princípio da Responsabilidade Única de forma proposital (e vale a pena)



Para lembrar

Um pedido, uma fila de responsáveis: cada um resolve o que sabe e repassa o resto — sem ninguém precisar saber tudo sozinho.

Referências

Refactoring Guru — refactoring.guru/pt-br/design-patterns/chain-of-responsibility

GAMMA, Erich et al. Padrões de projeto. Bookman, 2000.

LARMAN, Craig. Utilizando UML e padrões. 3. ed. Bookman, 2007.